



Linux Kernel Teaching

This is a collection of lectures and labs Linux kernel topics. The lectures focus on theoretical and Linux kernel exploration.

The labs focus on device drivers topics and they resemble “howto” style documentation. Each topic has two parts:

- a walk-through the topic which contains an overview, the main abstractions, simple examples and pointers to APIs
- a hands-on part which contains a few exercises that should be resolved by the student; to focus on the topic at hand, the student is presented with a starting coding skeleton and with in-depth tips on how to solve the exercises

This content is based on the [Operating Systems 2](#) course from the Computer Science and Engineering Department, the Faculty of Automatic Control and Computers, University POLITEHNICA of Bucharest.

You can get the latest version at <http://github.com/linux-kernel-labs>.

To get started build the documentation from the sources after installing docker-compose on you host:

```
cd tools/labs && make docker-docs
```

then point your browser at **Documentation/output/labs/index.html**.

Alternatively, you can build directly on the host (see tools/labs/docs/Dockerfile for dependencies):

```
cd tools/labs && make docs
```

Lectures

- Sisteme de operare 2
 - The team
 - Where do we stand?
 - Resources
 - Community
 - Grading
 - Obiectivele cursului
 - Obiectivele laboratorului/temelor
 - Cursuri necesare
 - Despre curs
 - Despre curs (2)
 - Despre laborator
 - Despre teme
 - Despre teme (2)
 - Bibliografie curs
 - Bibliografie laborator
- Introduction
 - Lecture objectives:
 - Basic operating systems terms and concepts
 - User vs Kernel
 - Typical operating system architecture
 - Monolithic kernel
 - Micro kernel
 - Micro-kernels vs monolithic kernels
 - Address space
 - User and kernel sharing the virtual address space
 - Execution contexts
 - Multi-tasking
 - Preemptive kernel
 - Pageable kernel memory
 - Kernel stack
 - Portability
 - Asymmetric MultiProcessing (ASMP)
 - Symmetric MultiProcessing (SMP)
 - CPU Scalability
 - Overview the of Linux kernel
 - Linux development model
 - Maintainer hierarchy
 - Linux source code layout
 - Linux kernel architecture

- arch
- Device drivers
- Process management
- Memory management
- Block I/O management
- Virtual Filesystem Switch
- Networking stack
- Linux Security Modules
- System Calls
 - Lecture objectives:
 - Linux system calls implementation
 - System call table
 - System call parameters handling
 - Virtual Dynamic Shared Object (VDSO)
 - Accessing user space from system calls
- Interrupts
 - Lecture objectives
 - What is an interrupt?
 - Exceptions
 - Hardware
 - Programmable Interrupt Controller
 - Advanced Programmable Interrupt Controller
 - Interrupt Descriptor Table
 - Interrupt handler address
 - Stack of interrupt handler
 - Interrupt handler execution
 - Returning from an interrupt
 - Nested interrupts and exceptions
 - Deferrable actions
- Symmetric Multi-Processing
 - Lecture objectives:
 - Synchronization basics
 - Linux kernel concurrency sources
 - Atomic operations
 - Disabling preemption (interrupts)
 - Spin Locks
 - Cache coherency in multi-processor systems
 - Optimized spin locks
 - Process and Interrupt Context Synchronization

- [Mutexes](#)
- [Per CPU data](#)
- [Memory Ordering and Barriers](#)
- [Read Copy Update \(RCU\)](#)
- [Debugging](#)
 - [Lecture objectives:](#)
 - [Decoding an oops/panic](#)
 - [Decoding an oops](#)
 - [addr2line](#)
 - [objdump](#)
 - [gdb](#)
 - [Kernel panic](#)
 - [List debugging](#)
 - [Memory debugging](#)
 - [DEBUG_SLAB](#)
 - [KASAN](#)
 - [Comparison between memory debugging tools](#)
 - [kmemcheck](#)
 - [DEBUG_PAGEALLOC](#)
 - [SLUB_DEBUG](#)
 - [Kmemleak](#)
 - [Lockdep checker](#)
 - [Other tools](#)

Labs

- [Infrastructure](#)
- [Introduction](#)
 - [Lab objectives](#)
 - [Keywords](#)
 - [About this laboratory](#)
 - [Documentation](#)
 - [Source code navigation](#)
 - [cscope](#)
 - [Kscope](#)
 - [LXR Cross-Reference](#)
 - [SourceWeb](#)
 - [Debugging](#)
 - [gdb \(Linux\)](#)
 - [Getting a stack trace](#)

- Documentation
- Exercises
 - Remarks
 - 1. Booting the virtual machine
 - 2. Adding and using a virtual disk
 - 3. GDB and QEMU
 - 4. GDB spelunking
 - 5. Cscope spelunking
- Kernel modules
 - Lab objectives
 - Overview
 - An example of a kernel module
 - Compiling kernel modules
 - Loading/unloading a kernel module
 - Debugging
 - objdump
 - addr2line
 - minicom
 - netconsole
 - Printk debugging
 - Dynamic debugging
 - Dyndbg Options
 - KDB: Kernel debugger
 - Exercises
 - 0. Intro
 - 1. Kernel module
 - 2. Printk
 - 3. Error
 - 4. Sub-modules
 - 5. Kernel oops
 - 6. Module parameters
 - 7. Proc info
 - Extra Exercises
 - 1. KDB
 - 2. PS Module
 - 3. Memory Info
 - 4. Dynamic Debugging
 - 5. Dynamic Debugging During Initialization
- Kernel API

- Lab objectives
- Overview
- Accessing memory
- Contexts of execution
- Locking
- Preemptivity
- Linux Kernel API
 - Convention indicating errors
 - Strings of characters
 - printk
 - Memory allocation
 - lists
 - Spinlock
 - mutex
 - Atomic variables
 - Use of atomic variables
 - Atomic bitwise operations
- Exercises
 - 0. Intro
 - 1. Memory allocation in Linux kernel
 - 2. Sleeping in atomic context
 - 3. Working with kernel memory
 - 4. Working with kernel lists
 - 5. Working with kernel lists for process handling
 - 6. Synchronizing list work
 - 7. Test module calling in our list module
- Character device drivers
 - Laboratory objectives
 - Overview
 - Majors and minors
 - Data structures for a character device
 - struct file_operations
 - inode and file structures
 - Implementation of operations
 - Registration and unregistration of character devices
 - Access to the address space of the process
 - Open and release
 - Read and write
 - ioctl
 - Waiting queues

- Exercises
 - 0. Intro
 - 1. Register/unregister
 - 2. Register an already registered major
 - 3. Open and close
 - 4. Access restriction
 - 5. Read operation
 - 6. Write operation
 - 7. ioctl operation
- Extra Exercises
 - ioctl with messaging
 - ioctl with waiting queues
 - O_NONBLOCK implementation
- I/O access and Interrupts
 - Lab objectives
 - Background information
 - Accessing the hardware
 - Request access to I/O ports
 - Accessing I/O ports
 - 5. Accessing I/O ports from userspace
 - Interrupt handling
 - Requesting an interrupt
 - Implementing an interrupt handler
 - Locking
 - Interrupt statistics
 - Further reading
 - Serial Port
 - Parallel port
 - Keyboard controller
 - Linux device drivers
 - Exercises
 - 0. Intro
 - Keyboard driver
 - 1. Request the I/O ports
 - 2. Interrupt handling routine
 - 3. Store ASCII keys to buffer
 - Reading the data register
 - Interpreting the scancode
 - 4. Store characters to the buffer

- 5. Reset the buffer
- Deferred work
 - Lab objectives
 - Background information
 - Softirqs
 - Tasklets
 - Timers
 - Locking
 - Workqueues
 - Kernel threads
 - Further reading
 - Exercises
 - 0. Intro
 - 1.Timer
 - 2. Periodic timer
 - 3. Timer control using ioctl
 - 4. Blocking operations
 - 5. Workqueues
 - 6. Kernel thread
 - 7. Buffer shared between timer and process
- Block Device Drivers
 - Lab objectives
 - Overview
 - Register a block I/O device
 - Register a disk
 - struct gendisk structure
 - struct block_device_operations structure
 - Request queues
 - Create and delete a request queue
 - Useful functions for processing request queues
 - Requests for block devices
 - Create a request
 - Finish a request
 - Process a request
 - struct bio structure
 - Create a struct bio structure
 - Submit a struct bio structure
 - Wait for the completion of a struct bio structure
 - Initialize a struct bio structure
 - How to use the content of a struct bio structure

- Free a `struct bio` structure
 - Set up a request queue at `struct bio` level
- Further reading
- Exercises
 - 0. Intro
 - 1. Block device
 - 2. Disk registration
 - 3. RAM disk
 - 4. Read data from the disk
 - 5. Write data to the disk
 - 6. Processing requests from the request queue at `struct bio` level
- File system drivers (Part 1)
 - Lab objectives
 - Virtual Filesystem (VFS)
 - The general file system model
 - superblock
 - Localization:
 - inode
 - Localization:
 - file
 - Localization:
 - dentry
 - Register and unregister filesystems
 - Functions `mount`, `kill_sb`
 - Superblock in VFS
 - The `struct super_block` structure
 - Superblock operations
 - The `fill_super()` function
 - Buffer cache
 - Functions and useful macros
 - Further reading
 - Exercises
 - myfs
 - 1. Register and unregister the myfs file system
 - 2. Completing myfs superblock
 - 3. Initialize myfs root inode
 - 4. Test myfs mount and unmount
 - minfs

- 1. Registering and unregister the minfs file system
 - 2. Completing minfs superblock
 - 3. Creating and destroying minfs inodes
 - 4. Initialize minfs root inode
 - 5. Testing of minfs mount and unmount
- File system drivers (Part 2)
 - Lab objectives
 - Inode
 - The inode structure
 - Inode operations
 - Getting an inode
 - Superoperations
 - inode_operations
 - The file structure
 - Regular files inodes
 - Regular files inode operations
 - Address space operations
 - Dentry structure
 - Dentry operations
 - Directory inodes operations
 - Creating an inode
 - Creating a directory
 - Creating a link
 - Creating a symbolic link
 - Deleting a link
 - Deleting a directory
 - Searching for an inode in a directory
 - Iterating through entries in a directory
 - Bitmap operations
 - Further reading
 - Exercises
 - myfs
 - 1. Directory operations
 - Testing
 - 2. File operations
 - Testing
 - minfs
 - 1. Iterate operation
 - Testing

- 2. Lookup operation
 - Testing
 - 3. Create operation
 - Testing
- Networking
 - Lab objectives
 - Overview
 - Networking in user space
 - Linux networking
 - The struct socket structure
 - Operations on the socket structure
 - Creation
 - Closing
 - Sending/receiving messages
 - The struct socket fields
 - The struct proto_ops structure
 - The struct sock structure
 - The struct sk_buff structure
 - Conversions
 - netfilter
 - netcat
 - Further reading
 - Exercises
 - 1. Displaying packets in kernel space
 - 2. Filtering by destination address
 - 3. Listening on a TCP socket
 - 4. Accepting connections in kernel space
 - 5. UDP socket sender
- Memory mapping
 - Lab objectives
 - Overview
 - Structures used for memory mapping
 - struct page
 - struct vm_area_struct
 - struct mm_struct
 - Device driver memory mapping
 - Further reading
 - Exercises

- 1. Mapping contiguous physical memory to userspace
- 2. Mapping non-contiguous physical memory to userspace
- 3. Read / write operations in mapped memory
- 4. Display memory mapped in procfs
- Linux Device Model
 - Overview
 - sysfs
 - Basic Structures in Linux Devices
 - The kobject structure
 - Buses
 - Devices
 - Drivers
 - Classes
 - Hotplug
 - Plug and Play
 - PNP bus
 - PNP operations
 - Add driver
 - Exercises
 - 0. Intro
 - 1. Bus implementation
 - 2. Add type and version device attributes
 - 3. Add del and add bus attributes
 - 4. Register the bex misc driver
 - 5. Register misc device in the bex_misc probe function
 - 6. Monitor uevent notifications

Useful info

- Virtual Machine Setup
 - Starting the VM
 - Connecting to the VM
 - Connecting a debugger to the VM kernel
 - Rebuild the kernel image
- Contributing to linux-kernel-labs
 - Repository structure
 - Building the documentation
 - Creating a contribution
 - Forking the repository
 - Creating a pull request

- [Making changes to a Pull Request](#)

Next ➞

© Copyright The kernel development community

Built with [Sphinx](#) using a [theme](#) provided by [Read the Docs](#).